Simon Fraser University
CMPT 295 – D100
Summer 2025
Wednesday, July 23, 2025
Midterm 2 - SOLUTION
Out of 45 marks

This examination has 14 pages.

Read each question carefully before answering it.

- No textbooks, cheat sheets, calculators, computers, cell phones or other materials/devices may be used.
- All assembly code must be x86-64 assembly code using ATT syntax and executable on our *target machine* unless stated otherwise.
- **Always comment your code.**
- The marks for each question are given in [ ]. Use this to manage your time:
  - One (1) mark corresponds to one (1) minute of work.
  - Do not spend more time on a question than the number of marks assigned to it.

Good luck!

1. [Total: 7 marks] Imagine we create a new Instruction Set Architecture (ISA) defined as following:

   - Memory model: 2^m x n = 2^{16} × 16 bits
   - Word size: 16 bits
   - 8 registers (called `r0` to `r7`)
   - Memory addressing mode:
     - Indirect: General syntax is `(`r_b`)` and meaning (or effect) is `M[R[`r_b`]]`
     - Base + displacement: General syntax is `Imm(`r_b`)` and meaning (or effect) is `M[Imm + R[`r_b`]]`
     - Registers can also be used as operands
   - Machine code instruction format:

Opcode	Dest	Src	Imm

     We managed to express all 64 instructions in the instruction set (IS) of this ISA using this one format, which is 1 word in length. You can think of `Imm` as a signed offset, which can have a positive or a negative value as well as zero (0).
   - Here are some assembly instructions in the instruction set (IS) of this ISA:
     - `PLUS (r0), r1` Meaning: `R[r1] <- M[R[r0]] + R[r1]]`
     - `SAVE r1, 8(r2)` Meaning: `M[8 + R[r2]] <- R[r1]`

     NOTE: There is a typo in the above instruction. The instruction should be:
     - `SAVE r1, 2(r2)` Meaning: `M[2 + R[r2]] <- R[r1]` where 2 is within the range of Imm (answer to Question 1. a.). 8 was not in the range of Imm.
     - So, the Question 1 a. has been cancelled, i.e., Midterm 2 is now out of 42 marks (as opposed to 45).

a. [3 marks] What is the range of values `Imm` can have in the machine code instructions in the instruction set (IS) of this ISA? Show your work.

Answer: $[-2^{4-1} \ldots +2^{4-1}-1] = [-2^3 .. +2^3 -1] = [-8 .. +7]$

Show my work: The machine code instruction format is 1 word in length -> 16 bits

To uniquely identify 8 registers we need 3 bits -> 3 bits (`Dst`) + 3 bits (`Src`)

The Opcode is 6 bits since we have 64 instructions in this IS -> 6 bits

So: 16 bits – 6 bits (`opcode`) – 3 bits (`Dest`) – 3 bits (`Src`) = 4 bits

Therefore, the `Imm` field in the machine code instruction format is 4 bits in length.

The range of signed values 4 bits can produce is $[-2^{4-1} \ldots +2^{4-1}-1]$

b. [2 marks] What is the memory address resolution of this ISA?

Answer: 16 bits

Lecture 24 Slide 13: memory address resolution is the smallest addressable memory "chunk" -> screen shot

Marking scheme: 2 marks for correct answer (there are no other answers)

-0.5 if units are not mentioned

c. [2 marks] What can be stored in the registers of this ISA? Note that "bits", "bytes" and "binary numbers" are not the answer! Be specific!

Answer: The registers are 16 bits in length (1 word), therefore

- memory addresses, which are 16 bits in length,
- data of data types up to 16 bits in length (not `Imm`), and
- machine code instructions, which are 16 bits in length.

can be stored in the registers of this ISA.

NOTE: The ISA does not name any data types (no `char`, `short`, `int`, `long`, `float`, `double`). Care not to mix your ISAs.

---

2. [3 marks] What is the size of the memory model of our ***target machine***? Show your work.

Answer: 2^{64} x 8 bits OR 2^{64} x 1 bytes (B)

Show work: Lecture 24 Slide 9: Memory size (of memory model) -> 2^m x n bits

On our target machine, since the memory addresses are 64 bits in length, m = 64 bits.

Since our target machine is a byte-addressable machine i.e., its memory address resolution is 8 bits, n = 8 bits.

Memory size (of memory model) -> 2^{64} x 8 bits

Marking scheme: 1 mark for correct answer and 2 marks for showing proper work.

-0.5 if units are not mentioned

3. [3 marks] Consider the following x86-64 assembly version of the recursive function `countOnesR(unsigned int x)` seen in our Lecture 21:

```
    .globl countOnesR
countOnesR:
    xorl    %eax,  %eax
    testq   %rdi,  %rdi
    je      done
    pushq   %rbx
    movq    %rdi,  %rbx
    andl    $1,    %ebx
    shrq    %rdi
    call    countOnesR
    addq    %rbx,  %rax
    popq    %rbx
done:
    ret
```

In Lecture 21, we were given two possible test cases, namely:

- Test Case 1: Input: x = 0
- Test Case 2: Input: x = 7

Create Test Case 3 by filling in the blanks:

- Test Case 3:
  - Input: x = _____
  - Expected result: _____

Possible Solution:

  - Input: x = 3
  - Expected result: 2

  This function counts the number of 1's in the binary representation of x.

4. [Total marks: 14] Consider the following C array declaration and initialization:

```
#define N 16
char A[N] = {1,-2,3,-4,-4,3,-2,1,-1,2,-3,4,4,-3,2,-1};
char target;
```

and the incomplete x86-64 assembly code for the function called `findTarget` below. This function uses linear search to search for `target` in the array `A`. It returns 1 if the `target` is found, otherwise it returns 0.

`findTarget` has three arguments:

1. `A`: the memory address of the first byte of the array A.
2. `N`: the size of the array. You can assume that N is an `unsigned int`.
3. `target`: the value we are looking for in the array A.

```
1.      .globl  findTarget
2.  # array A -> %rdi, N -> %esi, target -> %dl
3.  findTarget:
4.      endbr64
5.      testl   %esi, %esi              # N == 0?
6.                                      # yes, then target cannot be found!
7.      leal    -1(%rsi), %eax          # no, then compute N-1
8.                                      # compute memory address of first …
                                        # … byte beyond A: A+(N-1)+1 -> %rax
9.      jmp     loop
10. next_element:
11.                                     # compute address of next element of A
12.     cmpq    %rax, %rdi              # reached end of A?
13.     je      target_not_found        # yes, then target not found!
14. loop:
15.                                     # A[i] == target?
16.     jne      next_element           # no, then go to next element of A
17.     movl     $1, %eax               # yes, then target found and …
                                        # … set result value to 1
18.     ret
19. target_not_found:
20.                                     # most space efficient way of …
                                        # … setting result value to 0
21.     ret
```

Considering all that has been stated so far, answer the following questions:

a. [10 marks] There are five (5) x86-64 assembly instructions that are missing in the function `findtarget`: at lines 6, 8, 11, 15 and 20. Below, write each of the missing x86-64 assembly instructions beside its associated line number such that the function `findtarget` executes as described and expected. Make sure the instructions you write do satisfy their matching comment.

Line 6: `je target_not_found` **OR** `jz target_not_found`

Line 8: `leaq 1(%rdi, %rax), %rax`

OR `leaq 1(%rax, %rdi), %rax`

Line 11: `addq $1, %rdi` **OR** `incq %rdi`

Line 15: `cmpb (%rdi), %dl` **OR** `cmpb %dl, (%rdi)`

Line 20: `xorl %eax, %eax` **OR** `subl %eax, %eax`

No other instructions are accepted.

➔ 2 marks each

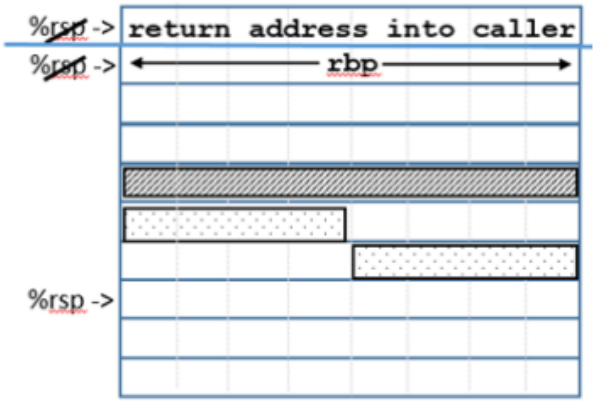
b. [4 marks] Why is the argument `target` stored in the register `%dl`? List two distinct reasons.

1. Reason 1: Because `target` is the third argument to the function and therefore must be stored in either `%rdx/%edx/%dx/%dl` (as opposed to `%rdi`, `%rsi`, ...). -> 2 marks
2. Reason 2: Because `target` is of `char` data type so it has a length of 8 bits and therefore must be stored in `%dl` (as opposed to `%rdx/%edx/%dx`). -> 2 marks

5. [Total marks: 18] Consider the incomplete x86-64 assembly code for the function called `mysteryFunction` below and the incomplete drawing of its stack diagram.

```
1     .globl  mysteryFunction
2  mysteryFunction:
3     endbr64
4
5     subq    $48, %rsp
6     movq    %fs:40, %rax
7     movq    %rax, 32(%rsp)
8     xorl    %eax, %eax
9     leaq    12(%rsp), %rbp
10    leaq    24(%rsp), %r11
11    movabsq <8 chars>, %rax  # These 8 chars are "PQRSTUV\0"
12    movq    %rax, 12(%rsp)
13    movw    $32, 20(%rsp)
14    movq    %r11, %rdi
15    xorl    %eax, %eax
16    movb    $7, 23(%rsp)
17    pushq   %r11
18    call    gets@PLT
19
20    movq    %r11, %rsi
21    movq    _______, %rdi
22    call    strcmp@PLT
23    testl   %eax, %eax
24    sete    %al
25    movq    _______(%rsp), _______
26    xorq    %fs:40, %rcx
27    jne     .L5
28    addq    _______, %rsp
29    popq    %rbp
30    ret
31 .L5:
32    call    __stack_chk_fail@PLT
```

Stack Diagram



Considering all of the above, answer the questions on the following pages.

a. [8 marks] First, hand trace the code, which has been replicated below, and in doing so fill in the missing instructions at Line 4 and Line 19 and complete the instructions at Line 21, Line 25 and Line 28. Do this directly in the code below.

In answering this question, you can make use of the partial stack diagram, but do not concern yourself with completing it. This comes in the next question.

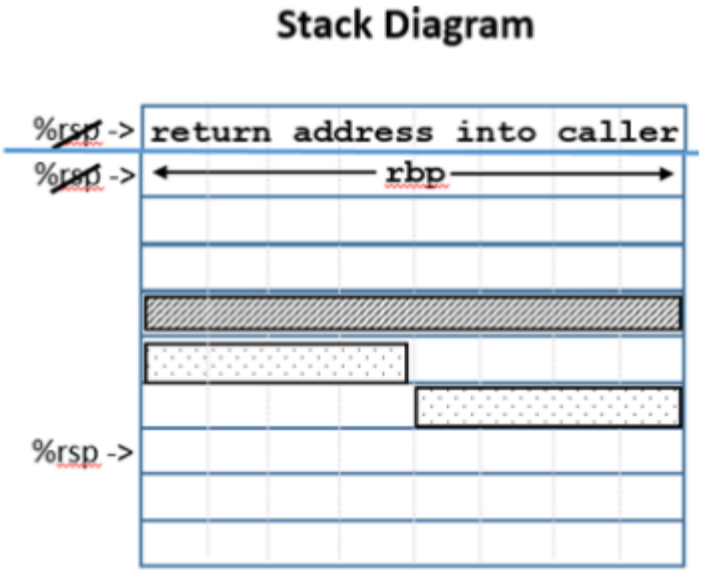
```
1       .globl  mysteryFunction
2   mysteryFunction:
3       endbr64
4
5       subq    $48, %rsp
6       movq    %fs:40, %rax
7       movq    %rax, 32(%rsp)
8       xorl    %eax, %eax
9       leaq    12(%rsp), %rbp
10      leaq    24(%rsp), %r11
11      movabsq <8 chars>, %rax  # These 8 chars are "PQRSTUV\0"
12      movq    %rax, 12(%rsp)
13      movw    $32, 20(%rsp)
14      movq    %r11, %rdi
15      xorl    %eax, %eax
16      movb    $7, 23(%rsp)
17      pushq   %r11
18      call    gets@PLT
19
20      movq    %r11, %rsi
21      movq    _______, %rdi
22      call    strcmp@PLT
23      testl   %eax, %eax
24      sete    %al
25      movq    _______(%rsp), _______
26      xorq    %fs:40, %rcx
27      jne     .L5
28      addq    _______, %rsp
29      popq    %rbp
30      ret
31  .L5:
32      call    __stack_chk_fail@PLT
```

Answer:

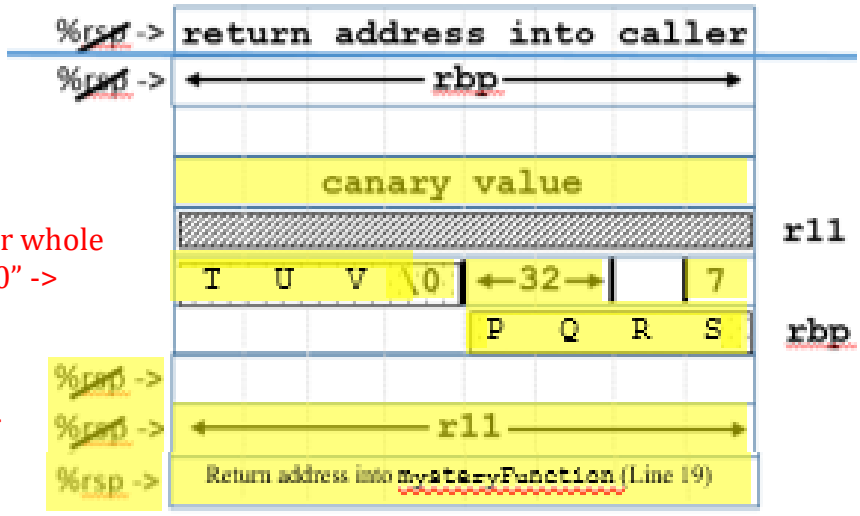
```
    .globl  mysteryFunction
mysteryFunction:
    endbr64
    pushq   %rbp                <- 2 marks
    subq    $48, %rsp
    movq    %fs:40, %rax
    movq    %rax, 32(%rsp)
    xorl    %eax, %eax
    leaq    12(%rsp), %rbp
    leaq    24(%rsp), %r11
    movabsq <8 chars>, %rax   # The
    movq    %rax, 12(%rsp)
    movw    $32, 20(%rsp)
    movq    %r11, %rdi
    xorl    %eax, %eax
    movb    $7, 23(%rsp)
    pushq   %r11
    call    gets@PLT
    popq    %r11                <- 2 marks
    movq    %r11, %rsi
    movq    %rbp, %rdi          <- 1 mark
    call    strcmp@PLT
    testl   %eax, %eax
    sete    %al
    movq    32(%rsp), %rcx      <- 1 mark each
    xorq    %fs:40, %rcx
    jne     .L5
    addq    $48, %rsp           <- 1 mark
    popq    %rbp
    ret
.L5:
    call    __stack_chk_fail@PLT
```

Marking scheme: No other possible answers.

b. [8 marks] Once you have a complete function, hand trace it again until Line 18 (inclusively) and in doing so, complete the drawing of this function's partial stack diagram replicated below. As you do so, add all the details available to you to this stack diagram.



Answer:



1.5 marks for whole "PQRSTUV\0" ->

<- 1 mark

<- 1 mark for 32

<- 1 mark for 7

0.5 marks each ->

<- 1 mark

<- 1 mark

Marking scheme: No other possible answers.

c. [2 marks] Which mechanism(s) is/are used in the assembly code of our function in order to prevent malicious executable code from being loaded and executed?

Answer: Canary value <- 1 mark

endbr64 <- 1 mark

Marking scheme: No other possible answers.

This page was intentionally left blank. Use it if needed.

Table of x86-64 Registers

64-bit (quad)	32-bit (double)	16-bit (word)	8-bit (byte)		
63..0	31..0	15..0	15..8	7..0	
rax	eax	ax	ah	al	Return value
rbx	ebx	bx	bh	bl	Callee saved
rcx	ecx	cx	ch	cl	4th arg
rdx	edx	dx	dh	dl	3rd arg
rsi	esi	si		sil	2nd arg
rdi	edi	di		dil	1st arg
rbp	ebp	bp		bpl	Callee saved
rsp	esp	sp		spl	Stack pointer
r8	r8d	r8w		r8b	5th arg
r9	r9d	r9w		r9b	6th arg
r10	r10d	r10w		r10b	Caller saved
r11	r11d	r11w		r11b	Caller saved
r12	r12d	r12w		r12b	Callee saved
r13	r13d	r13w		r13b	Callee saved
r14	r14d	r14w		r14b	Callee saved
r15	r15d	r15w		r15b	Callee saved

Table of Powers of 2

Power of 2^x	Value	Power of 2^x	Value	
0	1			
1	2	-1	1/2	0.5
2	4	-2	1/4	0.25
3	8	-3	1/8	0.125
4	16	-4	1/16	0.0625
5	32	-5	1/32	0.03125
6	64	-6	1/64	0.015625
7	128	-7	1/128	0.0078125
8	256	-8	1/256	0.00390625
9	512	-9	1/512	0.001953125
10	1024	-10	1/1024	0.0009765625
11	2048			
12	4096			
13	8192			
14	16384			
15	32768			
16	65536			

Table of x86-64 Jumps

Instruction	Condition	Description
`jmp`	always	Unconditional jump
`je` / `jz`	`ZF`	Jump if equal / zero
`jne` / `jnz`	`~ZF`	Jump if not equal / not zero
`js`	`SF`	Jump if negative
`jns`	`~SF`	Jump if nonnegative
`jo`	`OF`	Jump if overflow
`jno`	`~OF`	Jump if not overflow
`jg` / `jnle`	`~(SF ^ OF) & ~ZF`	Jump if greater (signed >)
`jge` / `jnl`	`~(SF ^ OF)`	Jump if greater or equal (signed ≥)
`jl` / `jnge`	`SF ^ OF`	Jump if less (signed <)
`jle` / `jng`	`(SF ^ OF) | ZF`	Jump if less or equal (signed ≤)
`ja` / `jnbe`	`~CF & ~ZF`	Jump if greater (unsigned >)
`jae` / `jnb`	`~CF`	Jump if greater or equal (unsigned ≥)
`jb` / `jnae` / `jc`	`CF`	Jump if less (unsigned <)
`jbe` / `jna` / `jnc`	`CF | ZF`	Jump if less or equal (unsigned ≤)